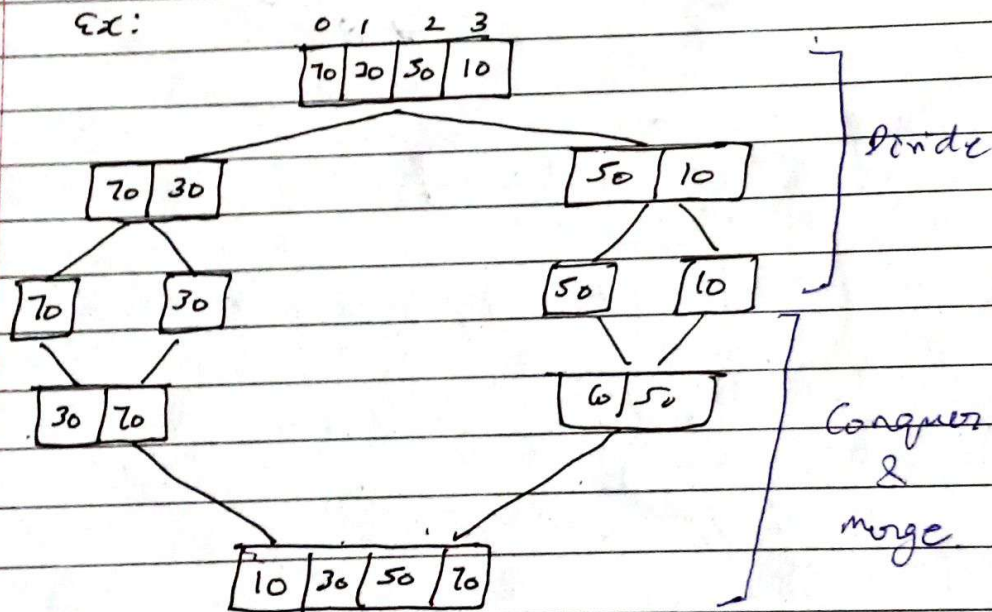


TASK-2Merge Sort

- It follows the Divide and Conquer approach.
- It works by recursively dividing the input array into two halves, recursively sorting the two halves and finally merging them back together to obtain the sorted array.

Time Complexity $\rightarrow O(N \log N)$

Ex:

Quick Sort

- It picks a element as pivot and partition the array around the picked point by placing the pivot in its correct position in sorted array.

- Choose the pivot $\rightarrow$ Partition the Array $\rightarrow$ Sort the array recursively
 $\downarrow$
 Base Case
 When single element is left

- Time Complexity $\rightarrow$ Worst Case $\rightarrow O(N^2)$
 Avg Case $\rightarrow O(N \log N)$

Ex:

0	1	2	3	4
5	2	4	1	3

pivot = 3

So,

2 1 3 5 4

next pivot

[2 1] 3 [5 4]

next pivot

 $\downarrow$

1, 2 3 4, 5

 $\therefore$ Ans = 1, 2, 3, 4, 5 Ans.DSA Practice Problems

Q- Merge Intervals : Overlapping.

Hint: Current and next interval ka element compare krna.

PAGE No.:

DATE:

Ex:

Input: $[[4, 7], [1, 4]]$

Output: $[[1, 7]]$

```
vector<vector<int>> merge (vector<vector<int>> &intervals,
```

```
sort(intervals.begin(), intervals.end());
```

```
vector<vector<int>> ans;
```

```
vector<int> current = intervals[0];
```

```
int i;
```

```
for (i=1; i<intervals.size(); i++) {
```

```
// overlapping
```

```
if (current[1] >= intervals[i][0]) {
```

```
current[1] = max (current[1], intervals[i][1]);
```

```
} // not overlapping (separate)
```

```
else
```

```
{
```

```
ans.push_back(current);
```

```
current = intervals[i];
```

```
}
```

```
ans.push_back(currentintervals[i]);
```

```
return ans;
```

```
}
```

Q- find the Duplicate Number

Input:

0	1	2	3	4
1	3	4	2	2

Output: 2

0	1	2	3	4
1	3	4	2	2

1

~~1 == 2 x~~

~~3 == 4 x~~

~~4 == 2 x~~

2 == 2 ✓

return 2 ✓

0	1	2	3	4
1	2	2	3	4

1 == 2 x

2 == 2 ✓

int k;

sort array;

for (i = 0; i < n - 1; i++)
{

if (nums[i] == nums[i + 1])
{

k = nums[i];

}
return k;

Q - Code Modification Challenge.

```
public class Sorter {
    public static void main (String args[]) {
        int C[] arr = {5, 2, 9, 1, 5, 6};
        int n = arr.length;
```

```
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - 1 - i; j++) {
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
```

```
                }
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

Ques

- 1) Array ko descending mai sort krne
- 2) Sorting ke damian kitne swaps hue hain, count krke last mai print krke do.
- 3) agr pehle se array sorted hai toh optimize krke code ko

```
public class Sorter {
    public static void main (String args[]) {
        int C[] arr = {5, 2, 9, 1, 5, 6};
        int n = arr.length;
        bool swapped;
        int count = 0;
```

```

for (int i = 0; i < n - 1; i++) {
    for list swapped = false;
    for (int j = 0; j < n - 1 - i; j++) {
        if (arr[j] < arr[j + 1]) {
            int temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
            swapped = true;
            count++;
        }
    }
}

```

```

if (!swapped) {
    break;
}

cout << count;

```

Q- Tricky Code Analysis:

Case-1 Enhanced for loop Mutation:

```

public class Test1 {
    public static void main (String args) {
        int arr = {5, 10, 15};
        for (int x: arr) {
            x = x * 2;
        }
        println (arr[0]);
    }
}

```

Prog runarr = {^{0 1 2} 5, 6, 15}Output:

5

```
for (x=0; x<size; x++)
{
```

```
    x = x + 2;
```

```
}
```

When $x=0$, $x \Rightarrow 0$ $x=1$, $x=2$ $x=3$, $x=4$

arr ki value pe koi gar nhi padege as arr of
x nhi hai.

• Case 2: Array Initialization & Default Values

```
public class Test2 {
    public static void main (String args[]) {
        int arr [] = new int (3);
        soph (arr (1));
```

```
    }
```

```
}
```

Output

0

Prog run

int arr [] = new int (3)

soph (arr (1))

Case 3: Array Bounds Trap

```
public class Test3 {
```

```
    public static void main (String args[]) {
```

```
int[] arr = new int(0);
sopln(arr.length);
sopln(arr[0]);
```

↳
↳

Output

0

Index Out of Bounds

Dry run

```
int[] arr = new int(0);
sopln(arr.length);
sopln(arr[0]);
```

arr[0] → Index out of Bounds → Runtime error as array mei
koi value nhi hai khali hai.

Case 4: The Length Property vs Method

```
public class Test4 {
    psvm(String args[]) {
        int[] arr = {1, 2, 3};
        sopln(arr.length);
    }
}
```

Dry run

```
int[] arr = {1, 2, 3};
```

sopln(arr.length) → compile time error
as arr.length is not a
method.

arr.length() ✓

Case 5: Array Print Behaviour

```
public class Test5 {
    public static void main(String args[]) {
        char[] chars = {'a', 'b', 'c'};
        int[] nums = {1, 2, 3};
        println(chars);
        println(nums);
    }
}
```

Try run → No error.

Output:

abc
123

• Case 6: Post Increment in Arrays

```
public class Test6 {
    public static void main(String args[]) {
        int[] arr = {10, 20, 30};
        int i = 0;
        println(arr[i++]);
        println(arr[i]);
    }
}
```

Try run .

$i = 0$

~~ans(i+1)~~ $\rightarrow i = 1$ so first print then $i = 1$

So,

Output

10

20

• Case 7: 2D Array References

```
public class Test 7 {
    print (String args[]) {
        int [][] matrix = new int [2][2];
        matrix[0] = new int [3];
        sopln (matrix[0][2]);
        sopln (matrix[1][0]);
    }
}
```

Dry run

row 0 $\rightarrow (0, 0, 0)$

row 1 $\rightarrow$ null

Output:

0

Runtime error

matrix[0][2] $\rightarrow 0$

matrix[1][0] $\rightarrow$ Runtime Error.

Case 8: Out of Bounds on loop Condition

```
public class Test 8 {
    print (String args[]) {
```

```
int C() arr = {1, 2, 3};
for (int i = 0; i < 3; i++) {
    if (arr[i] == 2) {
        Sopl'n arr[i+1];
    }
}
```

Output
Index out of Bound

dry run

i = 0
1 == 3 X

i = 1
2 == 3 X

i = 2
3 == 3 ✓

Sopl'n arr[3] → Runtime error
Index out of Bound

Case 9 - Overwriting References

```
public class Test9 {
    p1'm (String args[]) {
        int C() a = {1, 2};
        int C() b = {3, 4};
        a = b;
        b[0] = 99;
    }
    Sopl'n (a, b);
}
```

By run :Output
99

$$a = b$$

$$1, 2 = 3, 4$$

$$1, 2 = 99, 4$$

$$99, 2 = 99, 4 \text{ as } a = b.$$

Case 10 : Collection Overload

import java.util. ArrayList ;

public class Test10 {

public static void main (String args []) {

ArrayList < Integer > list = new ArrayList < Integer > (10);

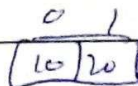
list.add(10);

list.add(20);

list.remove(10);

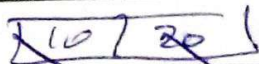
println(list);

}

Output :Index Out of Bound
ExceptionBy run :

remove (10)

→ Runtime error



→ [20] after 10 removed